

Міністерство освіти і науки України
Львівський національний університет імені Івана
Франка
Факультет прикладної математики та інформатики
Кафедра прикладної математики

Бакалаврська робота

**Розробка сервісу для стиснення зображень із
використанням триангуляції Делоне**

Студента групи ПМП-42:

Дулиби Олексія Олеговича

спеціальність 113 – прикладна математика

Науковий керівник:

к. ф.-м. н., доцент кафедри прикладної
математики

Макар Ігор Григорович

Зміст

Вступ	3
1 Формулювання задачі стиснення зображень	4
1.1 Класифікація методів стиснення зображень	4
1.2 Геометричні методи стиснення зображень	5
1.3 Постановка задачі	5
2 Властивості та побудова тріангуляції Делоне	7
2.1 Тріангуляція та умова Делоне	7
2.2 Метрики якості апроксимації	10
2.3 Алгоритм ітераційного процесу	12
3 Розробка сервісу для стиснення зображень	17
3.1 Засоби розробки та бібліотеки	17
3.2 Архітектура програми та структура модулів	18
3.3 Реалізація основних функцій програми	19
4 Тестування роботи сервісу	21
4.1 Тестування на зображенні з однорідними зонами	21
4.2 Тестування на фотографічному зображенні зі складною структурою	25
4.3 Порівняльний аналіз та узагальнення результатів	29
Висновки	32
Прикінцеві положення	34
Список використаних джерел	35

ВСТУП

Актуальність теми. У сучасних інформаційних системах обсяги графічних даних зростають досить швидко, що ставить перед розробниками завдання пошуку ефективних методів їх зберігання та передачі. Традиційне растрове представлення зображень, де кожному пікселю відповідає фіксована ділянка пам'яті, має значну надлишковість. Особливо це стосується зображень з великими однорідними областями та чіткими геометричними структурами.

Популярні стандарти стиснення з втратами, зокрема JPEG, використовують розбиття зображення на регулярні квадратні блоки. Такий підхід при високих коефіцієнтах стиснення призводить до появи специфічних візуальних артефактів на межах об'єктів. Альтернативою є геометричне представлення даних, де замість регулярної сітки використовується адаптивна тріангуляція, що підлаштовується під контури об'єктів. Використання тріангуляції Делоне дозволяє не лише стиснути дані, а й отримати аналітичну модель зображення, придатну для масштабування без втрати чіткості.

Мета дослідження — розробка та програмна реалізація алгоритму адаптивної тріангуляції для стиснення цифрових зображень, який забезпечує оптимальне наближення за метриками MSE та PSNR при мінімальній кількості опорних вузлів.

Об'єктом дослідження є процеси геометричної апроксимації та стиснення графічної інформації за допомогою нерегулярних сіток.

Предметом дослідження є ітераційні алгоритми подрібнення тріангуляції на основі аналізу локальних похибок апроксимації.

Для досягнення мети було поставлено такі **завдання**:

1. Проаналізувати сучасний стан методів растрового та векторного стиснення зображень.
2. Дослідити математичні властивості тріангуляції Делоне та методів апроксимації.
3. Розробити критерій адаптивного додавання вузлів у сітку на основі MSE.
4. Реалізувати програмний комплекс на мові Python для ітераційної побудови адаптивних сіток.

Розділ 1. ФОРМУЛЮВАННЯ ЗАДАЧІ СТИСНЕННЯ ЗОБРАЖЕНЬ

1.1 Класифікація методів стиснення зображень

Варто почати з самого визначення стиснення зображення. Стиснення зображення — це процес зменшення кількості даних, необхідних для представлення цифрового зображення, при збереженні прийнятної візуальної якості або точного відтворення оригіналу. Необхідність стиснення зумовлена тим, що растрове представлення зображення містить значну кількість надлишкової інформації: сусідні пікселі, як правило, мають близькі значення кольору, а великі однорідні ділянки зображення зберігаються повністю попиксельно, хоча для їх опису достатньо набагато менше даних.

Усі методи стиснення зображень поділяються на два принципово різних класи: стиснення без втрат та стиснення з втратами.

Стиснення без втрат (*lossless compression*) гарантує, що відновлене зображення є побітово ідентичним оригіналу. Такі методи застосовуються там, де будь-яке спотворення неприпустиме: медична діагностика, супутникові знімки, технічна документація. В основі методів стиснення без втрат лежить усунення статистичної надлишковості даних. Наприклад, алгоритм RLE (Run-Length Encoding) замінює послідовність однакових пікселів парою (значення, кількість повторень). Алгоритм LZ77, що використовується у форматі PNG, шукає повторювані послідовності байтів і замінює їх посиланнями на попередні входження. Типовий коефіцієнт стиснення для фотографічних зображень становить 1.5–3, що часто є недостатнім для практичних застосувань. До форматів без втрат належать PNG, BMP, TIFF, GIF.

Стиснення з втратами (*lossy compression*) дозволяє досягти значно вищих коефіцієнтів стиснення — від 5 до 100 і більше — за рахунок незворотного видалення частини інформації. Ключова ідея полягає у використанні особливостей людського зору: людське око менш чутливе до високочастотних деталей, незначних кольорових відхилень та дрібних текстурних елементів. Тому такі дані можна видалити або огрубити без помітного погіршення суб'єктивної якості зображення.

Сучасні методи стиснення з втратами поділяються на кілька підкласів залежно від математичного апарату, що використовується:

- **Трансформаційні методи** — зображення перетворюється з просторової області у частотну за допомогою математичних перетворень. Коефіцієнти, що відповідають високим частотам (дрібні деталі), відкидаються або огрублюються. Найбільш відомим представником є формат JPEG, що використовує дискретне косинусне перетворення (ДКП). Формат JPEG 2000 та WebP використовують вейвлет-перетворення, що забезпечує кращу якість при тому ж коефіцієнті стиснення.

- **Предикційні методи** — кожен піксель передбачається на основі сусідніх і зберігається лише похибка передбачення. Широко використовуються у відеокодеках H.264 та H.265 для міжкадрового стиснення.
- **Геометричні методи** — зображення апроксимується набором геометричних елементів: трикутників, прямокутників, кривих. До цього класу належить метод, що досліджується у даній роботі.

1.2 Геометричні методи стиснення зображень

Геометричні методи стиснення базуються на ідеї представлення зображення у вигляді набору геометричних елементів із заданим кольором. На відміну від растрового представлення, така модель природно адаптується до структури зображення.

Найбільш розповсюдженим геометричним підходом є триангуляційне стиснення: зображення апроксимується набором трикутників, кожен з яких характеризується координатами трьох вершин та значенням кольору — зазвичай середнім кольором оригінальних пікселів всередині трикутника. Для відновлення зображення достатньо знати лише ці параметри.

Принциповою перевагою геометричного підходу є **адаптивність**: алгоритм розміщує більше трикутників там, де зображення містить складні деталі та межі об'єктів, і великі трикутники на однорідних ділянках. Це дозволяє ефективно стискати зображення з чіткими межами: ілюстрації, карти, медичні знімки.

Об'єм закодованого представлення при K трикутниках визначається як:

$$S = K \cdot (3 \cdot 2 \cdot b_{\text{coord}} + 3 \cdot b_{\text{color}}), \quad (1.1)$$

де b_{coord} — кількість бітів на координату вершини, b_{color} — кількість бітів на канал кольору.

Для зображення 225×225 при $K = 20\,609$ трикутниках, $b_{\text{coord}} = 16$ біт (тип `uint16`), $b_{\text{color}} = 8$ біт отримуємо приблизно 27 КБ після стиснення — що у 6 разів менше, ніж оригінальний BMP файл розміром 156 КБ.

Серед інших геометричних підходів варто відзначити:

- **Апроксимація прямокутниками** — простіша у реалізації, але менш ефективна для зображень з похилими межами;
- **Апроксимація кривими Безьє** — точніше відтворює плавні контури, але вимагає складнішого кодування;
- **Вороноївське розбиття** — двоїсте до триангуляції Делоне, використовується в деяких системах стиснення текстур.

1.3 Постановка задачі

На основі проведеного аналізу предметної області та існуючих методів стиснення зображень сформулюємо задачу дипломної роботи.

Нехай задано растрове зображення

$$I = \{I(i, j) \mid 0 \leq i \leq M - 1, 0 \leq j \leq N - 1\}, \quad (1.2)$$

де $I(i, j) = (R, G, B)$ — значення пікселя у рядку i та стовпці j , M та N — висота і ширина зображення відповідно, кожен канал приймає цілі значення від 0 до 255.

Нехай задано також параметри алгоритму: $A_{\max}$ — максимальна допустима площа трикутника (пікселів²), що визначає початкову густоту сітки тріангуляції, та пороги якості PSNR* (у дБ) і MSE*, що визначають умову зупинки алгоритму.

Для оцінки якості апроксимації використовуються дві метрики. Середньо-квадратична похибка:

$$\text{MSE}(I, \hat{I}) = \frac{1}{MN} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \frac{1}{C} \sum_{c=1}^C \left[I_c(i, j) - \hat{I}_c(i, j) \right]^2, \quad (1.3)$$

де $\hat{I}$ — апроксимоване зображення, $C = 3$ — кількість каналів кольору (R, G, B), $I_c(i, j)$ та $\hat{I}_c(i, j)$ — значення c -го каналу пікселя оригіналу та апроксимації відповідно.

Пікове відношення сигнал/шум:

$$\text{PSNR}(I, \hat{I}) = 10 \cdot \log_{10} \frac{255^2}{\text{MSE}(I, \hat{I})}, \quad (1.4)$$

де 255 — максимально можливе значення пікселя для 8-бітного зображення. PSNR вимірюється в децибелах (дБ); більше значення відповідає кращій якості апроксимації.

Потрібно знайти множину точок $P = \{p_1, p_2, \dots, p_K\}$, де $p_k = (x_k, y_k)$, $0 \leq x_k \leq N - 1$, $0 \leq y_k \leq M - 1$, та відповідну тріангуляцію Делоне $\mathcal{T}(P)$ таку, що апроксимоване зображення $\hat{I}$, побудоване заповненням кожного трикутника $t \in \mathcal{T}(P)$ середнім кольором:

$$\hat{c}(t) = \frac{1}{|P_t|} \sum_{(i,j) \in P_t} I(i, j), \quad (1.5)$$

де P_t — множина пікселів всередині трикутника t , задовольняє умову якості:

$$\text{PSNR}(I, \hat{I}) \geq \text{PSNR}^* \quad \text{або} \quad \text{MSE}(I, \hat{I}) \leq \text{MSE}^*. \quad (1.6)$$

Для вибору трикутників, що потребують рефайнменту, вводиться оцінка якості трикутника:

$$\text{score}(t) = \bar{e}(t) \cdot |P_t|, \quad (1.7)$$

де $\bar{e}(t)$ — середня похибка пікселів всередині трикутника t , $|P_t|$ — кількість таких пікселів. Така оцінка враховує одночасно інтенсивність похибки та площу трикутника: великий трикутник з помірною похибкою може мати більший пріоритет, ніж малий трикутник з великою локальною похибкою.

Розділ 2. ВЛАСТИВОСТІ ТА ПОБУДОВА ТРІАНГУЛЯЦІЇ ДЕЛОНЕ

2.1 Тріангуляція та умова Делоне

Поняття тріангуляції. Тріангуляція множини точок на площині — це розбиття опуклої оболонки цієї множини на трикутники такі, що:

- вершинами кожного трикутника є точки з заданої множини;
- будь-які два трикутники або не перетинаються, або мають спільне ребро, або мають спільну вершину;
- об'єднання всіх трикутників покриває всю опуклу оболонку множини.

Для заданої множини P з n точок існує багато різних тріангуляцій. Кількість трикутників у будь-якій тріангуляції визначається формулою Ейлера і залежить лише від кількості точок та кількості точок на опуклій оболонці:

$$T = 2n - h - 2, \quad (2.1)$$

де T — кількість трикутників, n — загальна кількість точок, h — кількість точок на опуклій оболонці.

Серед усіх можливих тріангуляцій однієї множини точок тріангуляції відрізняються лише вибором діагоналей у чотирикутниках, що утворюються сусідніми парами трикутників. Операція заміни однієї діагоналі чотирикутника на іншу називається **переключенням ребра** (*edge flip*) і є базовою операцією переходу між різними тріангуляціями однієї множини точок.

Означення тріангуляції Делоне. Тріангуляція Делоне — це особливий вид тріангуляції, що задовольняє умову Делоне і є оптимальною за рядом геометричних критеріїв. Вперше описана радянським математиком Борисом Миколайовичем Делоне у 1934 році.

Умова Делоне. Тріангуляція $\mathcal{T}(P)$ множини точок P називається тріангуляцією Делоне, якщо для кожного трикутника $t \in \mathcal{T}(P)$ описане коло $C(t)$ не містить жодної точки з P у своєму внутрішньому просторі:

$$\forall t \in \mathcal{T}(P), \quad \forall p \in P \setminus V(t) : \quad p \notin \text{int}(C(t)), \quad (2.2)$$

де $V(t) = \{v_1, v_2, v_3\}$ — множина вершин трикутника t , $\text{int}(C(t))$ — відкритий внутрішній простір описаного кола.

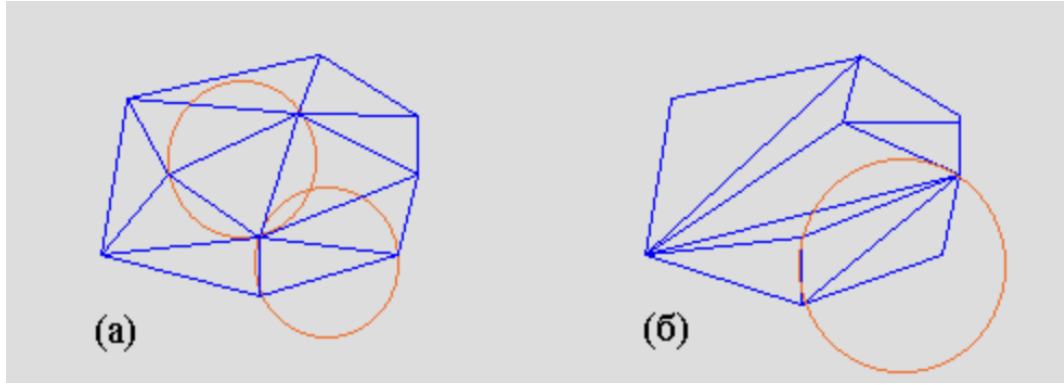


Рис. 2.1 – (а)Виконується умова Делоне (б)Не виконується умова Делоне

Умову Делоне можна перевіряти локально для кожного ребра. Нехай ребро e є спільним для двох трикутників t_1 та t_2 , що утворюють чотирикутник Q . Ребро e задовольняє умову Делоне тоді і лише тоді, коли четверта вершина t_2 не лежить всередині описаного кола t_1 . Це перевіряється через знак визначника:

$$\det \begin{pmatrix} x_1 - x_4 & y_1 - y_4 & (x_1 - x_4)^2 + (y_1 - y_4)^2 \\ x_2 - x_4 & y_2 - y_4 & (x_2 - x_4)^2 + (y_2 - y_4)^2 \\ x_3 - x_4 & y_3 - y_4 & (x_3 - x_4)^2 + (y_3 - y_4)^2 \end{pmatrix} > 0, \quad (2.3)$$

де $(x_1, y_1), (x_2, y_2), (x_3, y_3)$ — вершини трикутника t_1 обходом проти годинникової стрілки, (x_4, y_4) — четверта вершина. Якщо визначник від'ємний, ребро потрібно переключити.

Властивості тріангуляції Делоне. Тріангуляція Делоне має ряд важливих властивостей, що роблять її особливо придатною для задач апроксимації зображень.

Властивість 1. Максимізація мінімального кута. Серед усіх можливих тріангуляцій множини P тріангуляція Делоне максимізує мінімальний кут серед усіх трикутників:

$$\mathcal{T}_D(P) = \arg \max_{\mathcal{T}} \min_{t \in \mathcal{T}} \alpha_{\min}(t), \quad (2.4)$$

де $\alpha_{\min}(t)$ — найменший кут трикутника t . Ця властивість гарантує, що в тріангуляції не виникають вироджені вузькі трикутники. Для апроксимації зображень це критично: вузький трикутник покриває вузьку смугу пікселів і його середній колір погано представляє відповідну ділянку зображення.

Властивість 2. Єдиність. За умови загального положення точок (жодні чотири точки не є кокуновими) тріангуляція Делоне є єдиною для заданої множини P . Це гарантує детермінованість алгоритму: для тих самих вхідних даних завжди отримуємо той самий результат незалежно від порядку обробки точок.

Властивість 3. Зв'язок з діаграмою Вороного. Тріангуляція Делоне є двоїстим графом до діаграми Вороного тієї ж множини точок. Діаграма Вороного розбиває площину на клітини:

$$V(p_i) = \{x \in \mathbb{R}^2 \mid d(x, p_i) \leq d(x, p_j), \forall j \neq i\}, \quad (2.5)$$

де d — евклідова відстань. Ребра триангуляції Делоне з'єднують точки, чії клітини Вороного мають спільну межу.

Властивість 4. Локальність перевірки. Умову Делоне (2.2) можна перевіряти локально для кожного ребра незалежно від решти триангуляції. Це дозволяє будувати ефективні інкрементальні алгоритми, де нова точка вставляється і локально відновлюється умова Делоне без перебудови всієї триангуляції.

Алгоритми побудови триангуляції Делоне. Існує кілька класів алгоритмів побудови триангуляції Делоне, що відрізняються підходом та обчислювальною складністю.

Алгоритм Боуєра–Вотсона є найбільш відомим і простим у реалізації. Для кожної нової точки p знаходяться всі трикутники, описане коло яких містить p , вони видаляються, утворюючи порожнину, і замість них будуються нові трикутники з вершиною в p . Обчислювальна складність: $O(n^2)$ у найгіршому випадку та $O(n \log n)$ в середньому.

Інкрементальна вставка з переключенням ребер. Точки додаються по одній. Після вставки нової точки перевіряються всі суміжні ребра і виконуються переключення за критерієм (2.3) до відновлення умови Делоне. Обчислювальна складність: $O(n \log n)$ в середньому.

Алгоритм «розділяй і володарюй». Множина точок рекурсивно ділиться на дві половини, для кожної будується триангуляція, потім виконується злиття з відновленням умови Делоне вздовж межі розділення. Обчислювальна складність: $O(n \log n)$ у найгіршому випадку.

Якісна триангуляція Делоне. Для практичних застосувань базової триангуляції Делоне часто недостатньо — потрібно додатково контролювати якість трикутників. Алгоритм Рупперта (1995) та його вдосконалення Шьючуком вирішують цю задачу шляхом вставки додаткових точок — центрів описаних кіл «поганих» трикутників.

Трикутник вважається «поганим», якщо виконується хоча б одна умова:

- мінімальний кут менший за заданий поріг: $\alpha_{\min}(t) < \alpha^*$, де зазвичай $\alpha^* = 20$;
- площа трикутника перевищує задане максимальне значення: $S(t) > A_{\max}$.

Алгоритм Рупперта–Шьючука гарантує, що після завершення всі трикутники задовольняють обидві умови якості. При цьому кількість вставлених додаткових точок є оптимальною в тому сенсі, що вона не перевищує кількість точок у будь-якій іншій якісній триангуляції більш ніж у константу разів. Обчислювальна складність алгоритму становить $O(n \log n)$, де n — кількість точок у фінальній триангуляції.

Параметр $A_{\max}$ є ключовим для задачі стиснення зображень: він визначає початкову густоту сітки. Менше значення $A_{\max}$ призводить до більшої кількості дрібних трикутників на початку алгоритму і точнішої початкової апроксимації, але збільшує обчислювальні витрати. Більше значення дає грубішу початкову сітку, яка потім уточнюється адаптивно в зонах з великою похибкою.

2.2 Метрики якості апроксимації

Для об'єктивної оцінки якості апроксимації зображення використовуються кількісні метрики, що дозволяють порівнювати різні методи стиснення незалежно від суб'єктивного сприйняття. У даній роботі застосовуються дві взаємопов'язані метрики — середньоквадратична похибка MSE та пікове відношення сигнал/шум PSNR.

Середньоквадратична похибка (MSE). Середньоквадратична похибка (*Mean Squared Error*, MSE) є фундаментальною мірою відхилення між двома зображеннями. Вона показує середній квадрат різниці між значеннями пікселів оригінального та відновленого зображень.

Для одноканального (градації сірого) зображення розміром $M \times N$ MSE визначається як:

$$\text{MSE} = \frac{1}{MN} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} [I(i, j) - \hat{I}(i, j)]^2, \quad (2.6)$$

де $I(i, j)$ — значення пікселя оригінального зображення у рядку i та стовпці j , $\hat{I}(i, j)$ — значення відповідного пікселя апроксимованого зображення.

Для кольорового зображення з C каналами (у даній роботі $C = 3$ для моделі RGB) MSE обчислюється як середнє по всіх каналах:

$$\text{MSE} = \frac{1}{MNC} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} \sum_{c=1}^C [I_c(i, j) - \hat{I}_c(i, j)]^2, \quad (2.7)$$

де $I_c(i, j)$ та $\hat{I}_c(i, j)$ — значення c -го каналу пікселя оригіналу та апроксимації відповідно, $c \in \{R, G, B\}$.

Зведення різниці у квадрат у формулі (2.7) має два важливі наслідки. По-перше, від'ємні та додатні відхилення не компенсують одне одного — враховується лише абсолютна величина відхилення. По-друге, великі відхилення штрафуються непропорційно сильно: відхилення в 20 одиниць дає внесок у 400, тоді як відхилення в 10 одиниць — лише 100, тобто вчетверо менше при вдвічі меншому відхиленні.

Властивості MSE:

- $\text{MSE} \geq 0$ завжди;
- $\text{MSE} = 0$ тоді і лише тоді, коли $\hat{I} \equiv I$ (ідеальна апроксимація);
- менше значення MSE відповідає кращій якості апроксимації;
- MSE чутлива до яскравих локальних артефактів через квадратичний характер штрафу.

Окрім глобального MSE для всього зображення, у задачах адаптивного стиснення важливу роль відіграє **локальна карта похибок** — двовимірний масив

$e(i, j)$, де кожен елемент показує похибку апроксимації одного пікселя:

$$e(i, j) = \frac{1}{C} \sum_{c=1}^C [I_c(i, j) - \hat{I}_c(i, j)]^2. \quad (2.8)$$

Карта похибок (2.8) використовується для визначення зон зображення, де апроксимація є найменш точною і куди потрібно додати нові точки тріангуляції. Глобальний MSE є середнім значенням карти похибок:

$$\text{MSE} = \frac{1}{MN} \sum_{i=0}^{M-1} \sum_{j=0}^{N-1} e(i, j). \quad (2.9)$$

Пікове відношення сигналу до шуму (PSNR). Пікове відношення сигнал/шум (*Peak Signal-to-Noise Ratio*, PSNR) є найбільш широко вживаною метрикою якості зображень у задачах стиснення. PSNR виражається через MSE і вимірюється в децибелах (дБ).

Для зображення з глибиною кольору 8 біт на канал (значення пікселів від 0 до 255) PSNR визначається як:

$$\text{PSNR} = 10 \cdot \log_{10} \frac{L^2}{\text{MSE}}, \quad (2.10)$$

де $L = 2^8 - 1 = 255$ — максимально можливе значення пікселя.

Величина $L^2 = 65025$ у чисельнику є максимально можливою похибкою у квадраті — тобто відповідає «найгіршому» можливому пікселю, де оригінал має значення 0, а апроксимація 255 або навпаки. Відношення L^2/MSE показує, у скільки разів реальна похибка менша за найгіршу можливу.

Логарифм переводить це відношення у зручну логарифмічну шкалу, що краще відповідає суб'єктивному сприйняттю якості людиною. Множник 10 забезпечує зручний числовий діапазон значень.

Властивості PSNR:

- PSNR > 0 завжди (для MSE < L^2);
- більше значення PSNR відповідає кращій якості;
- PSNR $\rightarrow \infty$ при MSE $\rightarrow 0$ (ідеальна апроксимація без втрат);
- зростання PSNR на 6 дБ приблизно відповідає вдвічі меншій похибці.

Практична шкала значень PSNR для зображень наведена у таблиці 2.1.

Табл. 2.1 – Шкала якості за метрикою PSNR

PSNR, дБ	Якість	Характеристика
менше 25	Незадовільна	Помітні спотворення
25 — 30	Прийнятна	Слабкі артефакти
30 — 35	Хороша	Незначні спотворення
35 — 40	Висока	Важко помітні відхилення
більше 40	Відмінна	Практично без спотворень

Зв'язок між MSE та PSNR. З формули (2.10) можна виразити MSE через PSNR:

$$\text{MSE} = L^2 \cdot 10^{-\text{PSNR}/10}. \quad (2.11)$$

Таблиця 2.2 ілюструє відповідність між значеннями MSE та PSNR для 8-бітного зображення.

Табл. 2.2 – Відповідність значень MSE та PSNR

MSE	PSNR, дБ
1000.0	18.1
500.0	21.1
100.0	28.1
50.0	31.1
25.0	34.1
10.0	38.1
1.0	48.1

Вибір метрики як критерію зупинки. У задачі адаптивного стиснення зображень обидві метрики використовуються як критерій зупинки ітераційного алгоритму. Алгоритм продовжує додавати нові точки тріангуляції доки не буде виконана умова:

$$\text{PSNR}(I, \hat{I}) \geq \text{PSNR}^* \quad \text{або} \quad \text{MSE}(I, \hat{I}) \leq \text{MSE}^*, \quad (2.12)$$

де PSNR^* та MSE^* — задані порогові значення якості.

Перевага використання PSNR як критерію полягає в тому, що його логарифмічна шкала краще відповідає суб'єктивному сприйняттю якості людиною. MSE зручніша, коли потрібен точний контроль абсолютної похибки в одиницях яскравості. Можливість задати обидва пороги одночасно дозволяє гнучко керувати умовою зупинки залежно від конкретної задачі.

2.3 Алгоритм ітераційного процесу

Адаптивне стиснення зображень методом тріангуляції Делоне реалізується через ітераційний процес послідовного уточнення геометричної сітки. На кожній ітерації алгоритм аналізує поточну якість апроксимації, визначає зони з найбільшою похибкою та додає нові точки тріангуляції саме в цих зонах. Процес повторюється до досягнення заданого рівня якості або вичерпання ліміту ітерацій.

Ініціалізація. На початку роботи алгоритму формується початкова множина точок P_0 , що складається з восьми опорних точок — чотирьох кутів зображення та чотирьох середин його сторін:

$$P_0 = \{(0, 0), (W-1, 0), (0, H-1), (W-1, H-1), (\lfloor W/2 \rfloor, 0), (\lfloor W/2 \rfloor, H-1),$$

$$(0, \lfloor H/2 \rfloor), (W-1, \lfloor H/2 \rfloor)\}, (2.13)$$

де W та H — ширина і висота зображення відповідно.

Вибір кутів і середин сторін як початкових точок забезпечує коректне покриття всієї площі зображення з перших ітерацій та запобігає виникненню трикутників, що виходять за межі зображення.

Побудова триангуляції та рендеринг. На кожній ітерації k для поточної множини точок P_k будується триангуляція Делоне $\mathcal{T}(P_k)$ з обмеженням максимальної площі трикутника $A_{\max}$. Обмеження площі гарантує, що навіть на першій ітерації при малій кількості вхідних точок трикутники не будуть надмірно великими.

Після побудови триангуляції виконується **рендеринг** — кожен трикутник $t \in \mathcal{T}(P_k)$ заповнюється середнім кольором пікселів оригінального зображення, що лежать всередині нього:

$$\hat{c}(t) = \frac{1}{|P_t|} \sum_{(i,j) \in P_t} I(i,j), \quad (2.14)$$

де $P_t = \{(i,j) \mid (i,j) \in t\}$ — множина пікселів всередині трикутника t , $|P_t|$ — їх кількість, $I(i,j)$ — колір пікселя оригінального зображення.

Результатом рендерингу є апроксимоване зображення $\hat{I}_k$ того ж розміру, що і оригінал, де кожен піксель має колір трикутника, який його покриває.

Обчислення карти похибок. Після рендерингу обчислюється **карта похибок** — двовимірний масив $e_k(i,j)$, де кожен елемент показує локальну похибку апроксимації відповідного пікселя:

$$e_k(i,j) = \frac{1}{C} \sum_{c=1}^C [I_c(i,j) - \hat{I}_{k,c}(i,j)]^2, \quad (2.15)$$

де C — кількість каналів кольору, $I_c(i,j)$ та $\hat{I}_{k,c}(i,j)$ — значення c -го каналу пікселя оригіналу та апроксимації відповідно.

На основі карти похибок обчислюються глобальні метрики якості MSE та PSNR за формулами (2.7) та (2.10).

Критерій зупинки. Після обчислення метрик перевіряється критерій зупинки:

$$\text{PSNR}(I, \hat{I}_k) \geq \text{PSNR}^* \quad \text{або} \quad \text{MSE}(I, \hat{I}_k) \leq \text{MSE}^* \quad \text{або} \quad k \geq N_{\max}, \quad (2.16)$$

де PSNR^* та MSE^* — задані порогові значення якості, $N_{\max}$ — максимально допустима кількість ітерацій.

Якщо хоча б одна з умов (2.16) виконана, алгоритм завершує роботу і повертає поточне апроксимоване зображення $\hat{I}_k$ як результат. Якщо жодна умова не виконана, алгоритм переходить до кроку адаптації.

Адаптація: вибір найгірших трикутників. Для кожного трикутника $t \in \mathcal{T}(P_k)$ обчислюється оцінка якості на основі карти похибок:

$$\text{score}(t) = \bar{e}(t) \cdot |P_t|, \quad (2.17)$$

де $\bar{e}(t)$ — середня похибка пікселів всередині трикутника:

$$\bar{e}(t) = \frac{1}{|P_t|} \sum_{(i,j) \in P_t} e_k(i, j). \quad (2.18)$$

Оцінка (2.17) враховує одночасно інтенсивність похибки та площу трикутника. Це дозволяє справедливо порівнювати великі та малі трикутники: великий трикутник з помірно рівномірною похибкою може мати більший пріоритет, ніж малий трикутник з локальним піком похибки, оскільки розбиття великого трикутника дає більший приріст якості.

Відбираються N_{worst} трикутників з найбільшим значенням score :

$$T_{\text{worst}} = \arg \max_{t \in \mathcal{T}(P_k), |T_{\text{worst}}| = N_{\text{worst}}} \text{score}(t). \quad (2.19)$$

Адаптація: додавання нових точок. Для кожного трикутника $t \in T_{\text{worst}}$ у множину точок додається m нових точок. Нові точки розміщуються за трьома стратегіями.

1. Точка максимальної похибки. Знаходиться піксель всередині трикутника t з найбільшим значенням $e_k(i, j)$:

$$(i^*, j^*) = \arg \max_{(i,j) \in P_t} e_k(i, j). \quad (2.20)$$

Ця точка гарантовано потрапляє в зону найбільшого локального спотворення.

2. Центроїд трикутника. Додається геометричний центр трикутника:

$$c(t) = \frac{v_1 + v_2 + v_3}{3}, \quad (2.21)$$

де v_1, v_2, v_3 — вершини трикутника t . Центроїд забезпечує рівномірне розбиття трикутника на менші частини.

3. Рівномірно розподілені точки. Решта $m - 2$ точок розміщуються рівномірно всередині трикутника за допомогою барицентричних координат. Для кожної точки генеруються випадкові барицентричні координати $(\lambda_1, \lambda_2, \lambda_3)$ такі, що $\lambda_1 + \lambda_2 + \lambda_3 = 1$ та $\lambda_i \geq 0$:

$$p = \lambda_1 v_1 + \lambda_2 v_2 + \lambda_3 v_3, \quad (2.22)$$

де координати рівномірно розподілені по площі трикутника, що досягається перетворенням:

$$\lambda_1 = r_1, \quad \lambda_2 = r_2 - r_1, \quad \lambda_3 = 1 - r_2, \quad (2.23)$$

де $r_1 \leq r_2$ — дві впорядковані рівномірно розподілені випадкові величини на $[0, 1]$.

Після додавання нових точок виконується видалення дублікатів — точок, що збіглись внаслідок округлення або випадкового збігу координат. Оновлена множина точок:

$$P_{k+1} = \text{unique}(P_k \cup \Delta P_k), \quad (2.24)$$

де ΔP_k — множина нових точок, доданих на ітерації k .

Загальна схема алгоритму. Повний алгоритм адаптивної тріангуляції можна представити у вигляді наступної послідовності кроків:

Крок 1. Ініціалізація опорних точок. На початковому етапі формується множина P_0 , яка складається з 8 базових вузлів, що визначають межі області

Крок 2. Побудова початкової тріангуляції. На основі множини P_0 будується тріангуляція Делоне $\mathcal{T}(P_0)$. Кожному трикутнику присвоюється колір, що відповідає середньому значенню кольорів пікселів, які потрапили в його межі.

Крок 3. Ітераційний процес подрібнення. Для кожної ітерації

$k = 0, 1, 2, \dots, N_{\max}$ виконуються наступні дії:

- а) **Обчислення похибки:** Для кожного трикутника $t \in \mathcal{T}(P_k)$ розраховується локальна похибка апроксимації (MSE) між оригінальним зображенням та його поточним наближенням.
- б) **Перевірка умови зупинки:** Якщо загальне значення PSNR досягло цільового рівня або кількість точок перевищила ліміт — перехід до Кроку 5.
- в) **Пошук критичних елементів:** Відбирається підмножина трикутників T_{worst} з найбільшими значеннями локальної похибки.

Крок 4. Додавання нових вузлів. В межах кожного трикутника з T_{worst} додається нова точка (наприклад, у точку з максимальним значенням похибки або в геометричний центроїд). Оновлена множина точок $P_{k+1} = P_k \cup \Delta P_k$ передається на наступну ітерацію до Кроку 2.

Крок 5. Завершення та рендеринг. Фіксація фінальної сітки та формування вихідного стисненого представлення зображення.

Складність алгоритму. Обчислювальна складність однієї ітерації визначається кількома складовими. Побудова тріангуляції Делоне для n точок має складність $O(n \log n)$. Рендеринг та обчислення карти похибок виконуються за $O(MN)$, де MN — кількість пікселів. Пошук найгірших трикутників потребує $O(T \cdot \bar{A})$, де T — кількість трикутників, $\bar{A}$ — середня площа трикутника.

Загальна складність однієї ітерації:

$$O_{\text{iter}} = O(n \log n + MN), \quad (2.25)$$

де домінуючим є доданок $O(MN)$ для зображень великого розміру.

Розділ 3. РОЗРОБКА СЕРВІСУ ДЛЯ СТИСНЕННЯ ЗОБРАЖЕНЬ

3.1 Засоби розробки та бібліотеки

Програма реалізована мовою **Python** версії 3.9 і вище. Вибір Python зумовлений кількома факторами. По-перше, мова має розвинену екосистему бібліотек для наукових обчислень та обробки зображень, що дозволяє зосередитись на реалізації алгоритму, не витрачаючи зусиль на низькорівневі операції. По-друге, Python забезпечує зручну роботу з багатовимірними масивами даних через бібліотеку NumPy, що є критично важливим для ефективної обробки пікселів зображення. По-третє, простота синтаксису та інтерактивний режим виконання дозволяють швидко перевіряти та налагоджувати алгоритм.

У програмі використовуються такі зовнішні бібліотеки.

NumPy є основою всіх числових обчислень у програмі. Зображення зберігається як тривимірний масив форми $(H \times W \times 3)$ з типом `uint8`, де H і W — висота і ширина зображення відповідно. NumPy забезпечує ефективне обчислення різниць між масивами пікселів, побудову координатної сітки для растеризації та операції з множинами точок тріангуляції.

Pillow використовується виключно для введення та виведення даних. Функція `Image.open` завантажує зображення з диска, автоматично визначаючи формат за заголовком файлу, та конвертує його у модель RGB. Функція `Image.fromarray` перетворює numpy-масив назад у об'єкт зображення, після чого метод `save` зберігає результат у форматі PNG.

Бібліотека triangle реалізує алгоритм Рупперта–Шьючука для побудови якісної тріангуляції Делоне. Вона приймає на вхід масив координат точок та рядок параметрів, що задають обмеження на мінімальний кут і максимальну площу трикутника, і повертає словник з масивами вершин та індексів трикутників.

OpenCV використовується для растеризації трикутників — визначення множини пікселів, що належать кожному трикутнику. Функція `cv2.fillPoly` заповнює полігон на бінарній масці, що дозволяє швидко отримати індекси пікселів всередині трикутника.

Scikit-image надає стандартні реалізації метрик якості.

Функція `peak_signal_noise_ratio` обчислює PSNR, а `mean_squared_error` — MSE. Обидві функції приймають масиви типу `float64` та параметр `data_range`, що задає максимально можливе значення пікселя.

Matplotlib використовується для побудови графічних результатів. Бібліотека формує чотирипанельні рисунки для кожної ітерації алгоритму, що містять оригінальне зображення, тріангульовану апроксимацію, розподіл точок із тепловою картою похибок та графік зміни PSNR по ітераціях.

3.2 Архітектура програми та структура модулів

Програма побудована за модульним принципом і складається з трьох окремих файлів, кожен з яких відповідає за чітко визначену функціональну область. Такий підхід відповідає принципу єдиної відповідальності: кожен модуль вирішує одну конкретну задачу і не залежить від деталей реалізації інших модулів. Це спрощує супровід коду, дозволяє незалежно тестувати окремі компоненти та полегшує подальше розширення функціональності програми.

Модуль `triangulation_logic.py` є ядром програми і містить увесь алгоритмічний код. У ньому зосереджена вся математична логіка адаптивної тріангуляції: ініціалізація точок, побудова сітки, рендеринг, обчислення похибок та управління ітераційним процесом. Модуль не залежить від жодного іншого модуля програми і може використовуватись незалежно.

Модуль `visualization_utils.py` відповідає за всі графічні виводи програми. Він повністю відокремлений від алгоритмічної логіки і працює виключно з даними, що передаються йому ззовні. Такий підхід дозволяє змінювати спосіб візуалізації незалежно від алгоритму тріангуляції.

Модуль `main.py` є точкою входу до програми і виконує роль координатора. Він імпортує необхідні компоненти з двох інших модулів та організовує послідовність виконання всієї програми. Важливою особливістю є те, що всі параметри запуску задаються безпосередньо у верхній частині цього файлу у вигляді іменованих констант, що дозволяє змінювати конфігурацію без пошуку параметрів у глибині коду.

Взаємодія між модулями відбувається в одному напрямку: модуль `main.py` імпортує компоненти з `triangulation_logic.py` та `visualization_utils.py`, тоді як ці два модулі не мають між собою жодних прямих залежностей. Дані передаються між модулями у вигляді `numpy`-масивів та словників-знімків, що містять стан тріангуляції на кожній ітерації.

Вхідними даними програми є:

- шлях до файлу зображення (`IMAGE_PATH`) — підтримуються формати BMP, JPG, PNG;
- максимальна площа трикутника (`MAX_AREA`) — визначає початкову густоту сітки тріангуляції в пікселях²;
- поріг якості за PSNR (`PSNR_THRESHOLD`) — алгоритм зупиняється, коли PSNR досягає цього значення;
- поріг якості за MSE (`MSE_THRESHOLD`) — альтернативний або додатковий критерій зупинки;
- максимальна кількість ітерацій (`MAX_ITERATIONS`) — обмежує час виконання у разі, якщо поріг якості недосяжний;
- кількість трикутників для рефайнменту на одній ітерації (`WORST_TRI_PER_ITER`) — визначає, скільки найгірших трикутників обробляється за одну ітерацію;
- кількість нових точок на трикутник (`POINTS_PER_TRI`) — задає інтенсив-

ність згущення сітки в зонах з великою похибкою.

Вихідними даними програми є:

- стиснене зображення у форматі PNG у папці `result_duplom/` — назва файлу формується автоматично з імені вхідного файлу та суфікса `_triangulated`;
- чотирипанельні рисунки для кожної ітерації, що показують стан апроксимації та розподіл похибки на поточному кроці;
- підсумковий рисунок, що порівнює першу та останню ітерації і відображає криві збіжності метрик PSNR та MSE;
- статистика у консолі: розміри зображення, кількість точок та трикутників, значення PSNR і MSE, час виконання.

3.3 Реалізація основних функцій програми

Зчитування та збереження зображення. Зчитування зображення виконується функцією `load_image` у модулі `main.py`. Функція відкриває файл за вказаним шляхом за допомогою бібліотеки Pillow та конвертує зображення у модель RGB, якщо воно має іншу колірну модель — палітрову або з каналом прозорості. Після цього зображення перетворюється у numpy-масив форми $(H \times W \times 3)$ з типом `uint8`, де H і W — висота і ширина зображення відповідно. Тип `uint8` означає, що кожен канал пікселя зберігається як ціле число від 0 до 255.

Збереження результату виконується функцією `save_result_image`. Вона автоматично формує назву вихідного файлу з імені вхідного зображення та суфікса `_triangulated`, після чого зберігає numpy-масив у форматі PNG через Pillow. Формат PNG обрано тому, що він забезпечує стиснення без втрат — збережене зображення є побітово ідентичним масиву апроксимованих пікселів.

Ініціалізація точок та побудова тріангуляції. Початкова множина точок формується функцією `initialize_points`. Вона повертає вісім точок — чотири кути зображення та чотири середини його сторін. Вибір саме цих точок гарантує, що тріангуляція коректно покриває всю площу зображення з перших ітерацій та не виходить за його межі.

Побудова тріангуляції виконується функцією `triangulate_points`, яка передає масив точок у бібліотеку `triangle` з рядком параметрів якості. Параметри задають якісну тріангуляцію з мінімальним кутом трикутника не менше 20° , відповідність умові Делоне та обмеження максимальної площі трикутника значенням `MAX_AREA`. Функція повертає словник з двома масивами: координатами вершин та індексами трикутників.

Рендеринг апроксимованого зображення. Функція `render_triangulation` заповнює кожен трикутник середнім кольором пікселів оригінального зображення, що лежать всередині нього. Для визначення множини пікселів кожного трикутника використовується функція `cv2.fillPoly` з бібліотеки OpenCV, яка заповнює бінарну маску за координатами вершин трикутника. Отримана маска дозволяє вибрати з оригінального зображення лише ті пікселі, що належать по-

точному трикутнику, і обчислити їх середній колір окремо для кожного каналу RGB. Один буфер маски повторно використовується для всіх трикутників, що дозволяє уникнути зайвого виділення пам'яті під час циклу.

Обчислення карти похибок та метрик. Функція `compute_error_map` обчислює попиксельну карту похибок як середній квадрат різниці між каналами оригіналу та апроксимації. Результатом є двовимірний масив форми $(H \times W)$, де кожен елемент відповідає середній квадратичній похибці відповідного пікселя по всіх трьох каналах. Ця карта є основою для пошуку зон зображення, де апроксимація є найменш точною.

Глобальні метрики якості обчислюються функціями `compute_psnr` та `compute_mse`. Перед обчисленням обидві функції переводять масиви пікселів у тип `float64`, щоб уникнути переповнення при відніманні значень типу `uint8`. Самі обчислення виконуються стандартними функціями бібліотеки `scikit-image`.

Пошук найгірших трикутників та додавання точок. Функція `find_worst_triangles` обчислює оцінку якості для кожного трикутника як добуток середньої похибки на кількість пікселів всередині нього та повертає n трикутників з найбільшим значенням цієї оцінки. Такий спосіб оцінювання враховує одночасно інтенсивність похибки та площу трикутника, що дозволяє справедливо порівнювати трикутники різного розміру.

Функція `add_points_in_worst_triangles` додає нові точки у кожен з відібраних трикутників за трьома стратегіями: точка з максимальною локальною похибкою, центроїд трикутника та рівномірно розподілені точки за барицентричними координатами. Після додавання всіх нових точок виконується видалення дублікатів, що могли виникнути внаслідок округлення координат.

Клас `AdaptiveTriangulation`. Клас `AdaptiveTriangulation` об'єднує всі описані функції в єдиний ітераційний процес. Конструктор класу приймає зображення та параметри алгоритму, зокрема максимальну площу трикутника, порогові значення PSNR і MSE, максимальну кількість ітерацій та параметри рефайнменту.

Основним методом класу є `run`, який виконує цикл ітерацій. На кожній ітерації послідовно викликаються побудова тріангуляції, рендеринг, обчислення метрик та перевірка критерію зупинки через метод `quality_ok`. Якщо критерій не виконано, відбувається оновлення множини точок. Після кожної ітерації стан програми зберігається у внутрішньому списку `history` у вигляді словника, що містить поточні точки, результат тріангуляції, відрендероване зображення, карту похибок та значення PSNR і MSE. Після завершення циклу метод `run` повертає цей список, який використовується модулем візуалізації для побудови графічних результатів.

Розділ 4. ТЕСТУВАННЯ РОБОТИ СЕРВІСУ

У даному розділі наведено результати практичного тестування розробленої програми адаптивної тріангуляції Делоне. Для всебічної оцінки алгоритму було обрано два тестових зображення, що принципово відрізняються за своєю структурою.

Перше зображення містить великі однорідні області з чіткими межами між ними — саме такий тип зображень є найбільш сприятливим для тріангуляційного стиснення, оскільки однорідні зони покриваються кількома великими трикутниками, а точки концентруються лише на межах об'єктів.

Друге зображення є фотографічним і містить дрібні деталі, плавні градієнти та складні текстури. На такому зображенні алгоритм працює в значно складніших умовах: однорідних зон майже немає, тому для досягнення того самого рівня PSNR потрібно значно більше точок тріангуляції.

Таке порівняння дозволяє виявити сильні та слабкі сторони розробленого алгоритму та визначити типи зображень, для яких метод є найбільш ефективним.

4.1 Тестування на зображенні з однорідними зонами

Перший експеримент проводився на тестовому зображенні розміром 225×225 пікселів. Це зображення характеризується наявністю великих за площею однорідних зон, чіткими контурами об'єктів та відсутністю складних текстур чи дрібного хаотичного шуму. Такий тип зображень є базовим для перевірки ефективності адаптивних алгоритмів апроксимації, оскільки головна задача полягає в суттєвій економії кількості точок усередині однорідних областей при збереженні високої точності на межах розподілу яскравості.

Зображення зберігалось у форматі BMP, його розмір на диску становив 156 КБ. Наведені параметри запуску програми дозволяють об'єктивно оцінити, як алгоритм мінімізує обчислювальні ресурси на спрощених геометричних структурах.

Вхідні параметри та налаштування алгоритму. Перед запуском обчислювального процесу для першого експерименту було визначено базову конфігурацію програми та задано такі вхідні параметри:

1. `Input_image = "image.bmp"` — вхідне тестове зображення у форматі BMP без стиснення, розміром 225×225 пікселів, яке виступає еталоном для побудови карти локальних похибок та адаптивної Делоне-тріангуляції.
2. `MAX_AREA = 200` — максимальна допустима площа трикутника. Цей параметр обмежує появу занадто великих геометричних елементів на першій ітерації та визначає щільність розбиття сітки Делоне.

3. $\text{PSNR_THRESHOLD} = 40.0$ дБ — цільове (порогове) значення пікового відношення сигналу до шуму, у разі досягнення якого алгоритм вважає апроксимацію успішною та автоматично завершує роботу.
4. $\text{MAX_ITERATIONS} = 8$ — максимальна кількість ітерацій адаптивного перерахунку сітки та додавання нових точок.
5. $\text{WORST_TRI_PER_ITER} = 150$ — кількість найгірших (з найбільшою похибкою) трикутників, які підлягають обов'язковому розщепленню та додаванню нових опорних точок протягом однієї ітерації.
6. $\text{POINTS_PER_TRI} = 20$ — кількість внутрішніх точок, які додаються всередині кожного окремого трикутника за одну ітерацію.



Рис. 4.1 – Вхідне тестове зображення (розмір 225×225 пікселів)

Динаміка якості по ітераціях. На початковому етапі алгоритм ініціалізував 8 опорних точок. Після першої ітерації стало очевидним, що наявність великих однорідних зон дозволяє утримувати малу кількість елементів сітки, проте значення PSNR на першій ітерації склало лише 22.44 дБ — цільовий (нормативний) рівень якості на початкових кроках алгоритму досягнутий не був. Це пояснюється тим, що початкова груба сітка ще не встигла адаптуватися до чітких геометричних меж об'єктів.

З кожною наступною ітерацією алгоритм концентрував нові точки виключно на контурах і межах перепаду яскравості, тоді як великі однорідні області залишалися покритими мінімальною кількістю трикутників. Через таку специфіку розподілу точок, зростання PSNR та падіння MSE відбувалося стабільно в міру того, як точніше описувалися межі фігур.

Алгоритм завершив роботу після 8 ітерацій, не досягнувши порогового значення $\text{PSNR} = 40$ дБ. Фінальна триангуляція містила всього 20329 точок та 73087 трикутників — це доводить високу ефективність розробленого методу для зображень такого класу. Загальний час виконання склав 231.11 секунд.

Динаміку зміни метрик якості PSNR та MSE, а також ріст кількості розрахункових точок протягом 8 ітерацій наведено у таблиці 4.1.

Табл. 4.1 – Динаміка метрик якості по ітераціях (зображення з однорідними зонами)

Ітерація	Кількість точок	PSNR, дБ	MSE
0	351	22.44	370.6
1	2979	30.01	64.8
2	5896	32.85	33.7
3	8804	34.93	20.9
4	11695	36.62	14.1
5	14578	37.43	11.7
6	17449	37.8	10.7
7	20329	38.49	9.2

Візуалізація процесу тріангуляції. На рисунках 4.2–4.7 наведено результати побудови тріангуляції Делоне на трьох ключових етапах обчислювального процесу: початкова апроксимація, проміжний стан сітки та фінальний результат роботи програми.

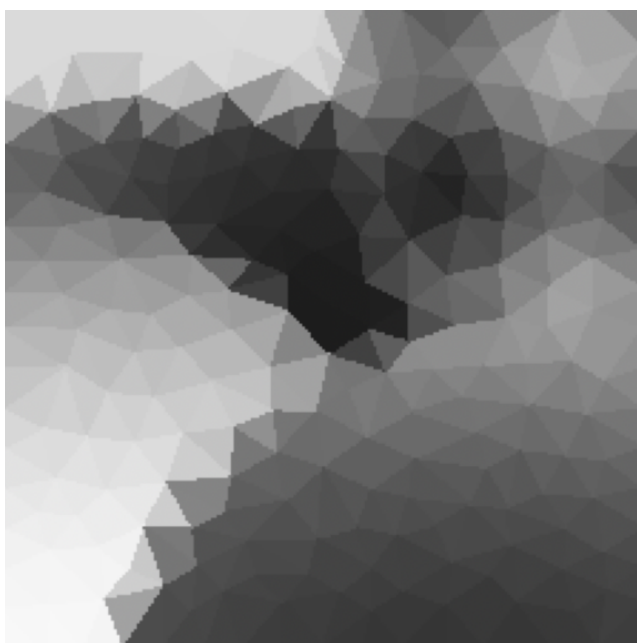


Рис. 4.2 – Апроксимація (Ітерація 0)

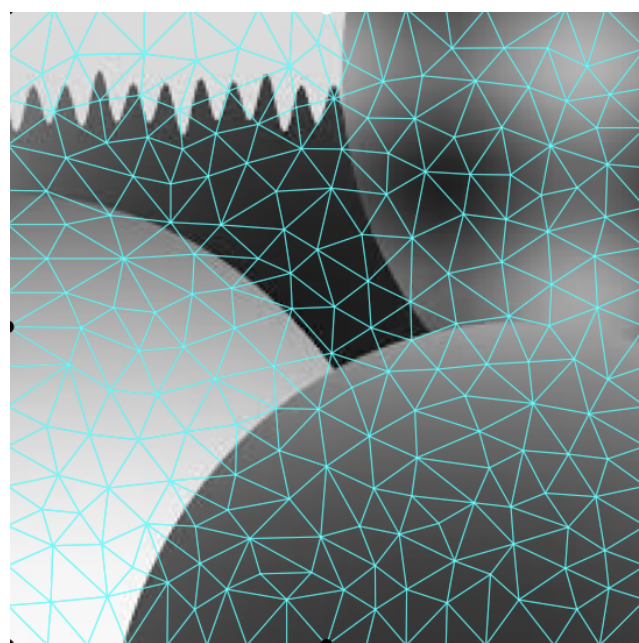


Рис. 4.3 – Сітка (Ітерація 0)

На початковій ітерації (рис. 4.2–4.3) апроксимація є вкрай грубою. Великі трикутники покривають значні за площею плоскі області, що призводить до сильного усереднення колірних тонів. Початкова сітка не враховує реальні межі об'єктів і побудована суто по опорних кутових точках.

Кольорове маркування обчислювальних вузлів відображає рівень локальної похибки MSE за допомогою палітри `hot`: чорний колір позначає стабільні зони з мінімальною помилкою, а проміжні червоні та помаранчеві відтінки фіксують ділянки із середнім відхиленням. Жовті вузли відповідають максимальній

локальній похибці на поточній ітерації та виступають тригерами, вказуючи алгоритму зони найгіршої якості, куди програма на наступному кроці примусово додасть нові точки для адаптивного ущільнення сітки Делоне.

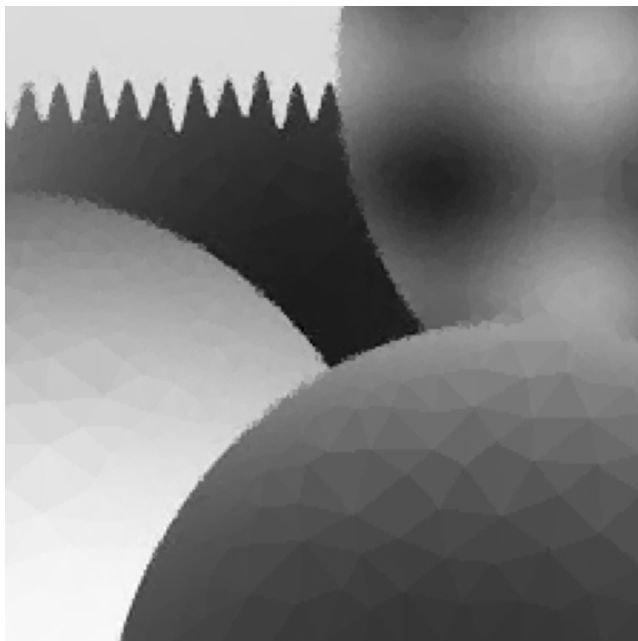


Рис. 4.4 – Апроксимація (Ітерація 3)

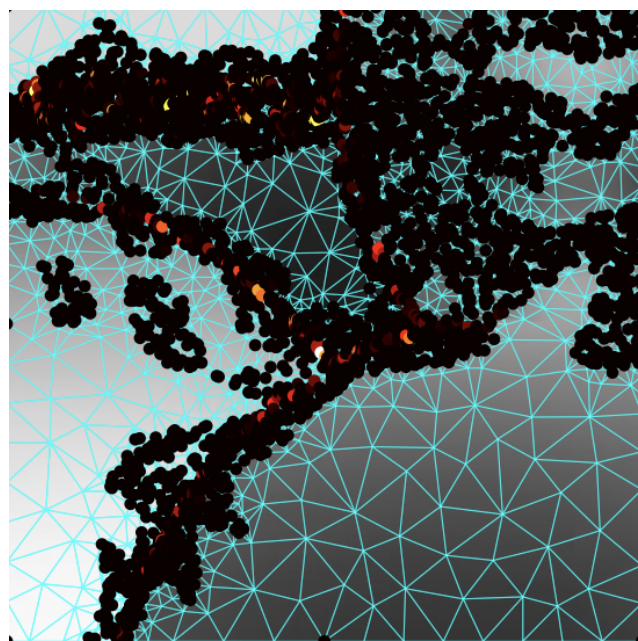


Рис. 4.5 – Сітка (Ітерація 3)

На проміжному етапі (рис. 4.4–4.5) чітко видно адаптивну природу розробленого алгоритму. Нові точки активно генеруються вздовж контурів геометричних об'єктів, де помилка апроксимації була найбільшою. Водночас середина великих однорідних зон залишається практично недоторканою, що демонструє локальну економію кількості елементів сітки.

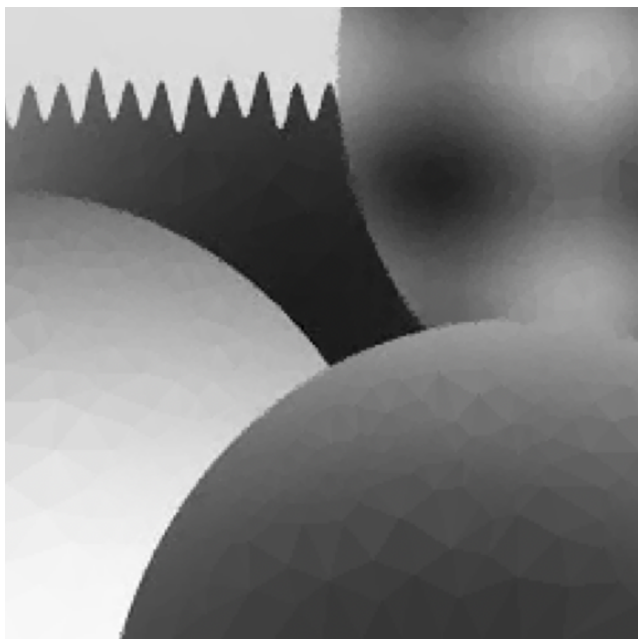


Рис. 4.6 – Апроксимація (Ітерація 7)

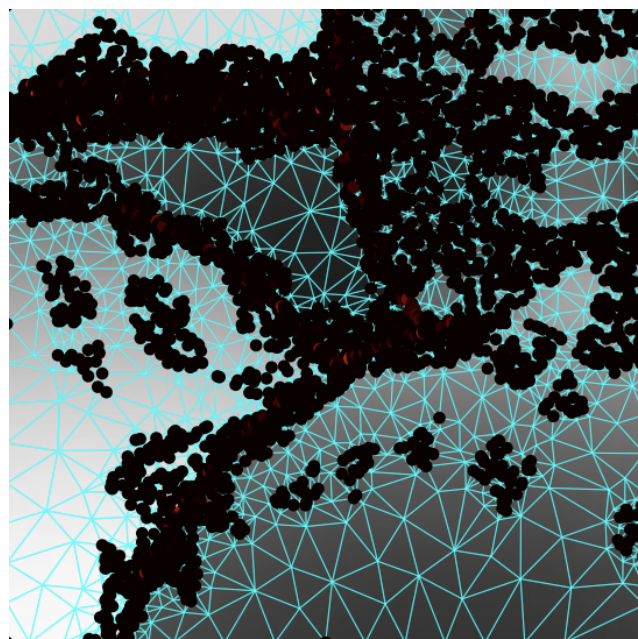


Рис. 4.7 – Сітка (Ітерація 7)

Фінальний результат (рис. 4.6–4.7) демонструє високу якість відновлення зображення. Обчислювальна сітка детально повторює контури об'єктів. При цьому «полігональний» ефект майже відсутній, оскільки всередині плоских однорідних областей колір апроксимується константою без значних похибок, що є найкращим сценарієм для кусково-лінійної моделі.

Результати стиснення. Стиснене зображення збережено у форматі PNG у папці `result_diplom`. Розмір вихідного файлу склав 26 КБ, що у 6 разів менше за об'єм оригінального файлу BMP. Отриманий коефіцієнт стиснення є дуже високим, що повністю обґрунтовується наявністю великої кількості фонових однорідних зон, які вдалося описати мінімальним числом трикутників. Досягнуте фінальне значення $PSNR = 38.49$ дБ підтверджує ефективність розробленого підходу для зображень такого класу.

4.2 Тестування на фотографічному зображенні зі складною структурою

Другий експеримент проводився на фотографічному зображенні розміром 671×1000 пікселів. На відміну від першого тестового зображення, це фото містить складну структуру: дрібні деталі, плавні градієнти кольорів, різноманітні текстури та відсутність великих однорідних областей. Саме такий тип зображень є найбільш складним для триангуляційного стиснення, оскільки алгоритм не може зекономити на внутрішніх точках однорідних зон і змушений рівномірно покривати всю площу густою сіткою.

Зображення зберігалось у форматі BMP, його розмір на диску становив 2016 КБ. Параметри запуску відрізнялись від першого експерименту — було збільшено максимальну площу трикутника та кількість найгірших трикутників на ітерацію, що зумовлено більшим розміром зображення.

Вхідні параметри та налаштування алгоритму. Перед запуском обчислювального процесу для другого експерименту було задано такі вхідні параметри:

1. `Input_image = "photo.bmp"` — вхідне деталізоване фотографічне зображення у форматі BMP без стиснення (розмір 671×1000 пікселів, зовнішній вигляд наведено на рис. 4.8).
2. `MAX_AREA = 200`
3. `PSNR_THRESHOLD = 40.0` дБ
4. `MAX_ITERATIONS = 6`
5. `WORST_TRI_PER_ITER = 300`
6. `POINTS_PER_TRI = 20`

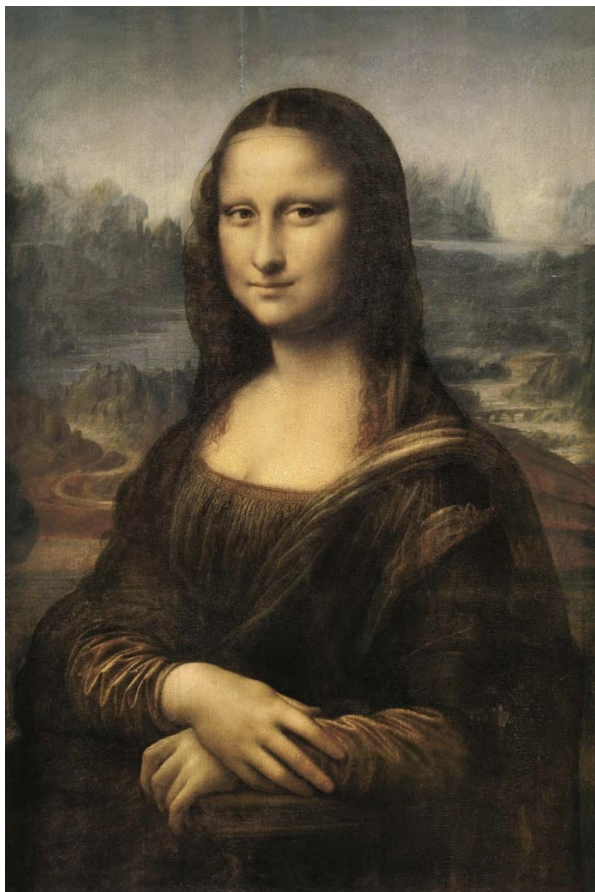


Рис. 4.8 – Вхідне фотографічне зображення зі складною структурою

Динаміка якості по ітераціях. На початковому етапі алгоритм ініціалізував 8 опорних точок. Після першої ітерації значення PSNR склало 25.25 дБ.

З кожною наступною ітерацією зростання PSNR відбувалось значно повільніше, ніж у першому експерименті. Якщо в першому випадку за перші три ітерації вдалося підвищити PSNR з 22.44 до 34.93 дБ, то тут за аналогічну кількість кроків — лише з 25.25 до 31.46 дБ. Це наочно демонструє, що фотографічні зображення є значно складнішими для триангуляційної апроксимації.

Алгоритм завершив роботу після 6 ітерацій, не досягнувши порогового значення $PSNR = 40.0$ дБ — зупинка відбулась через вичерпання ліміту ітерацій. Фінальне значення PSNR склало 32.30 дБ, що відповідає прийнятній якості апроксимації. Фінальна триангуляція містила 29 835 точок та 107510 трикутників. Загальний час виконання склав 2481 секунд.

Динаміку зміни метрик якості PSNR та MSE наведено у таблиці 4.2.

Табл. 4.2 – Динаміка метрик якості по ітераціях (зображення зі складною структурою)

Ітерація	Кількість точок	PSNR, дБ	MSE
0	8	25.25	193.92
1	5980	29.59	71.48
2	11960	30.83	53.72
3	17922	31.46	46.51
4	23880	31.93	41.66
5	29835	32.30	38.27

Візуалізація процесу тріангуляції. Графічні результати покрокової адаптації обчислювальної сітки Делоне наведено на рисунках 4.9–4.14.



Рис. 4.9 – Апроксимація (Ітерація 0)

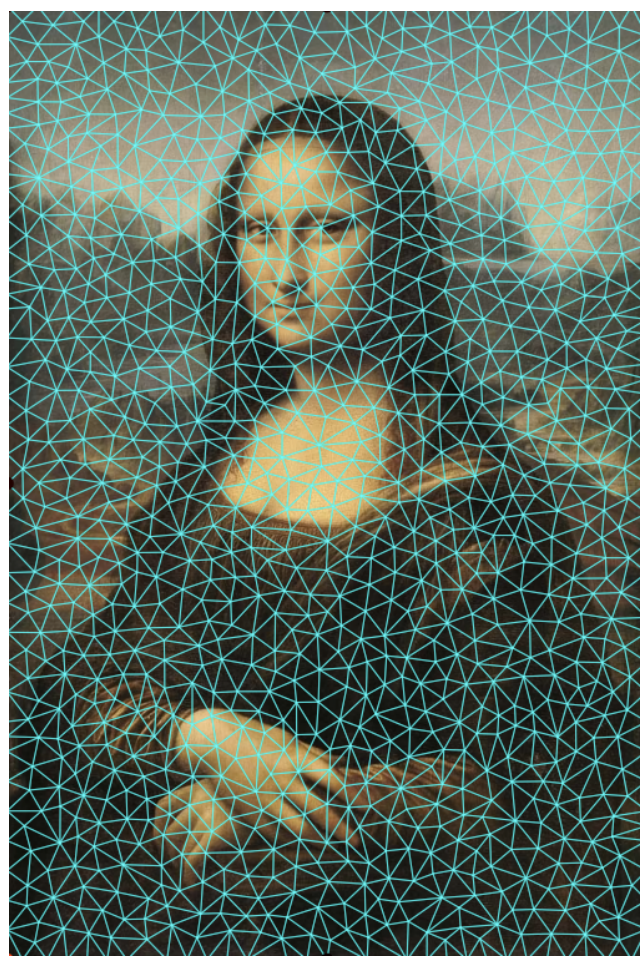


Рис. 4.10 – Сітка (Ітерація 0)

На початковому етапі (рис. 4.9–4.10) апроксимація є вкрай грубою — великі трикутники не здатні передати ані дрібні деталі, ані плавні переходи кольорів. Початкова сітка є рівномірною і не відображає жодних особливостей структури зображення.

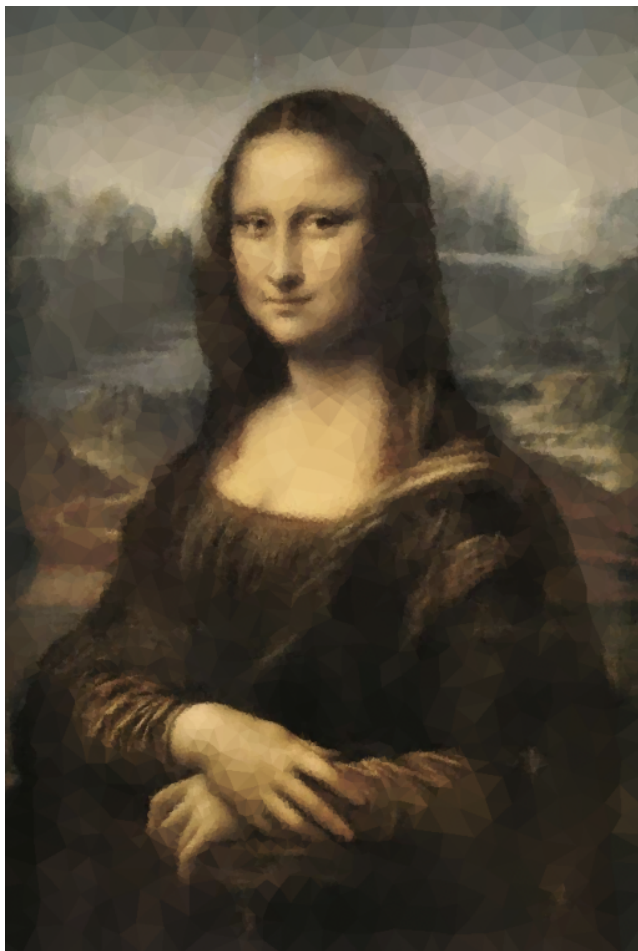


Рис. 4.11 – Апроксимація (Ітерація 2)

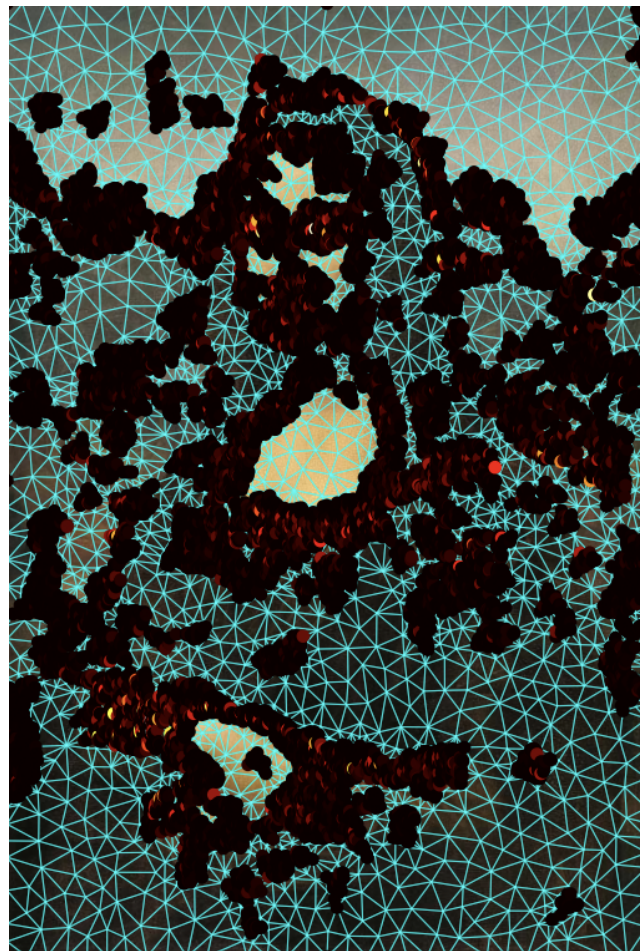


Рис. 4.12 – Сітка (Ітерація 2)

На проміжних кроках (рис. 4.11–4.12) спостерігається інтенсивне ущільнення контурів та частини заднього фону. На відміну від зображень з великими однорідними зонами, де сітка локалізується суто вздовж ліній контурів, тут точки розподіляються по всій площині кадру, намагаючись компенсувати похибку відтворення дрібних текстур.



Рис. 4.13 – Апроксимація (Ітерація 5)

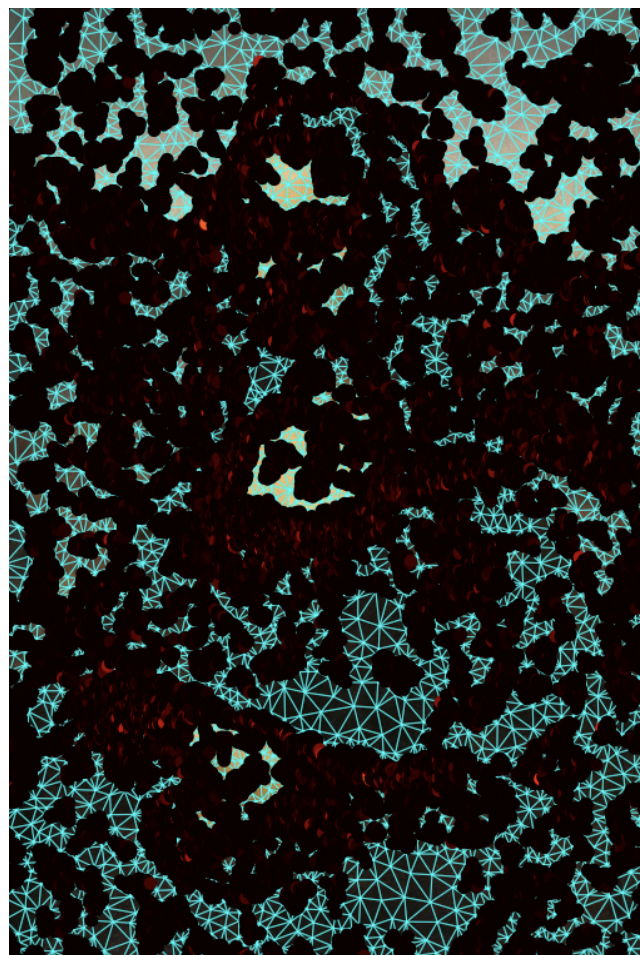


Рис. 4.14 – Сітка (Ітерація 5)

Фінальний стан моделі (рис. 4.13–4.14) демонструє високу щільність генерації трикутників.

Результати стиснення. Стиснене зображення збережено у форматі PNG у папці `result_duplom`. Розмір вихідного файлу склав 512 КБ, що у 4 разів менше за об'єм оригінального BMP файлу розміром 2016 КБ. Коефіцієнт стиснення є нижчим порівняно з першим експериментом, що пояснюється необхідністю використання значно більшої кількості трикутників для апроксимації складного фотографічного вмісту. Цільове значення $\text{PSNR} = 40.0$ дБ не було досягнуто — алгоритм зупинився через вичерпання ліміту ітерацій при фінальному значенні $\text{PSNR} = 32.30$ дБ, що відповідає хорошій якості апроксимації згідно зі шкалою, наведеною у розділі 2.

4.3 Порівняльний аналіз та узагальнення результатів

На основі комп'ютерних моделювань, проведених у підрозділах 4.1 та 4.2, було виконано комплексний порівняльний аналіз роботи розробленого алгоритму адаптивної тріангуляції Делоне для двох різних за структурою типів графічних

даних: зображення з однорідними зонами (Експеримент №1) та фотографічного зображення з високим рівнем деталізації та складними текстурами (Експеримент №2).

Для узагальнення та математичної оцінки динаміки кусково-лінійної апроксимації, ключові кількісні та якісні показники обчислювального процесу для обох експериментів зведено у порівняльну таблицю 4.3.

Табл. 4.3 – Зведені результати ефективності алгоритму для різних типів зображень

Показник ефективності	Експеримент №1 (Однорідні)	Експеримент №2 (Складні)
Розмірність вхідного файлу, пікс.	225 × 225	671 × 1000
Об'єм оригінального файлу (BMP)	156 КБ	2016 КБ
Загальна кількість ітерацій	8	6
Фінальна кількість точок сітки	20 329	29 835
Фінальна кількість трикутників	73 087	107 510
Загальний час виконання, сек.	231.11	2481.00
Фінальне значення $PSNR$, дБ	38.49	32.30
Фінальне значення похибки MSE	9.20	38.27
Об'єм стисненого файлу (PNG)	26 КБ	512 КБ
Коефіцієнт стиснення	6 разів	4 рази

Зіставлення отриманих кількісних даних дозволяє сформулювати такі науково-практичні висновки:

- Вплив інформаційної ентропії контенту на щільність сітки:** В Експерименті №1 алгоритм демонструє високу локальну економію елементів: нові вузли генеруються суто вздовж контурів, а середина плоских зон покривається великими трикутниками. В Експерименті №2 через відсутність однорідності алгоритм змушений рівномірно ущільнювати сітку по всій площі, що призвело до генерації 107 510 трикутників та збільшення часу розрахунку до 2481 секунд.
- Залежність швидкості збіжності від структури:** На контурних зображеннях ріст $PSNR$ відбувається значно швидше (зміна з 22.44 до 34.93 дБ за перші 3 кроки). На складних фотографічних структурах графік збіжності є пологим (зміна з 25.25 до 31.46 дБ за аналогічну кількість кроків), що обґрунтовує зупинку програми за лімітом ітерацій.
- Масштабованість обчислювальної складності та параметрична оптимізація:** Зі збільшенням просторової розмірності вхідної матриці пікселів (з 225 × 225 до 671 × 1000) спостерігається нелінійне зростання обчислювальної складності алгоритму, що безпосередньо відображається на загальному часі виконання програми (ріст з 231.11 до 2481.00 секунд). Для забезпечення стабільної динаміки адаптації сітки на великих масивах даних постає необхідність пропорційного масштабування внутрішніх параметрів керування. Зокрема, збільшення значення константи $WORST_TRI_PER_ITER$ (зі 150 до 300) є інженерно обґрунтованим кроком, який дозволяє за одну ітерацію розщеплювати більшу кількість дефектних трикутників, запобігаючи надмірному розтягуванню обчислюваль-

ного процесу в часі.

Порівняльний аналіз із JPEG. Для повного аналізу ефективності розробленого методу кусково-лінійної апроксимації доцільно порівняти його логіку та результати з класичним стандартом стиснення фотографій з втратами — JPEG. Фізико-математичні розбіжності між цими підходами згруповано за трьома критеріями:

1. **Природа математичного перетворення:** Алгоритм JPEG базується на дискретному косинусному перетворенні (DCT), яке розкладає зображення на матриці просторових частот блоками 8×8 пікселів, після чого виконується квантування (відсікання високих частот). Розроблений метод використовує геометричну декомпозицію простору за допомогою адаптивної тріангуляції Делоне та бере колір всередині трикутників середнім значенням його пікселів.
2. **Адаптивність до структури зображення та характер артефактів:** Головна перевага розробленого тріангуляційного алгоритму полягає в його динамічній адаптації під геометричну структуру кадру: точки концентруються спершу на межах перепаду яскравості, а коли похибка стає мінімальною переходить на інші елементи зображення, що дозволяє зберігати високу чіткість ліній і контурів без появи цифрового бруду. Натомість стандарт JPEG працює за фіксованою сіткою блоків 8×8 пікселів, через що при високих коефіцієнтах стиснення на контурах виникає ефект «дзвінка», а саме зображення розпадається на видимі квадрати. Водночас на ділянках із плавними колірними переходами розроблений метод створює помітний «полігональний» (low-poly) ефект, який є фундаментальною особливістю кусково-лінійної інтерполяції і потребує подальшого згладжування, тоді як JPEG на таких зонах забезпечує візуально м'якше відтворення.
3. **Оцінка ефективності:** Для складного фотографічного контенту (Експеримент №2) JPEG демонструє суттєво вищу ефективність. За ідентичних умов тестування алгоритм JPEG стиснув вихідне фото до **189 КБ**, що забезпечило значно менший об'єм файлу на диску порівняно з результатом нашого алгоритму (**512 КБ**), причому з цілком прийнятною візуальною якістю апроксимації. Це підтверджує, що JPEG на сьогодні є досконалим і важкодосяжним еталоном. Розроблений метод адаптивної тріангуляції на даному етапі показує високу конкурентоспроможність лише на зображеннях з великими однорідними областями (Експеримент №1), а для роботи з повноколірними складними фотографіями він усе ще потребує математичних вдосконалень.

Таким чином, розроблена система адаптивної тріангуляції Делоне є ефективною альтернативою частотним методам кодування для специфічного класу зображень з вираженою геометричною структурою.

ВИСНОВКИ

У кваліфікаційній роботі виконано комплексне дослідження, розробку та експериментальну оцінку ефективності алгоритму адаптивної тріангуляції Делоне для задач відновлення та кодування цифрових зображень різних структурних класів. Отримані науково-практичні результати дозволяють сформулювати такі підсумкові висновки:

Методика підходу та особливості реалізації. У роботі було запропоновано та програмно реалізовано ітераційну процедуру побудови адаптивної сітки скінченних елементів на основі тріангуляції Делоне. Основна ідея підходу полягає в динамічному аналізі карти локальних похибок за метрикою MSE всередині кожного окремого трикутника з подальшим вибіркоким розщепленням найбільш дефектних геометричних елементів на кожному кроці алгоритму. Науково обґрунтовано критерії адаптації сітки та забезпечено інтеграцію параметрів динамічного керування щільністю обчислювальних вузлів (таких як `MAX_AREA` та `WORST_TRI_PER_ITER`), що дозволило гнучко адаптувати програму до розширення вхідних графічних даних.

Наукова та практична цінність результатів. Наукова новизна роботи базується на дослідженні поведінки адаптивної тріангуляції Делоне на дискретних масивах пікселів з різним рівнем інформаційної ентропії. Практична цінність розробленого методу полягає у створенні дієвого програмного інструменту, що забезпечує високу чіткість збереження контурів та ліній розриву яскравості без появи блочних артефактів (ефекту «дзвінка»), які притаманні поширеним частотним методам стиснення. Результати експериментів підтвердили, що метод демонструє найвищу доцільність на графічних об'єктах із чіткими межами та розвиненими однорідними областями, забезпечуючи високу якість відновлення поля кольорів ($PSNR = 38.49$ дБ) при значному скороченні об'єму даних на диску (стиснення у 6 разів відносно формату BMP).

Вирішення поставлених у вступі завдань. Усі завдання, сформульовані на етапі планування наукового дослідження, було успішно виконано в повному обсязі. Зокрема, проведено аналіз існуючих підходів до побудови сіток, розроблено математичну модель адаптивного формування обчислювальних вузлів, збудовано працездатний програмний комплекс на мові Python та виконано серію комп'ютерних моделювань. Порівняльний аналіз із промисловим стандартом JPEG підтвердив, що для високоентропійних деталізованих сцен розроблений алгоритм на даному етапі поступається еталону за коефіцієнтом стиснення (розмір файлу 512 КБ проти 189 КБ у JPEG), проте забезпечує принципово інший, геометрично точний характер відновлення контурних елементів.

Прогноз щодо розвитку проблеми у майбутньому. Проведений аналіз обмежень розробленого підходу відкриває кілька перспективних напрямків для подальших наукових досліджень та оптимізації алгоритму.

По-перше, значне обчислювальне навантаження (що відобразилось у часі виконання 2481 секунда для великого фото) вказує на гостру потребу в ал-

горитмічній оптимізації процесу локального пошуку. Наступним кроком є впровадження ефективніших деревовидних структур даних (наприклад, *R*-дерев або *quad*-дерев) для швидкого пошуку трикутника, в який потрапляє аналізований піксель, що дозволить знизити часову складність алгоритму зліва направо.

По-друге, перспективним напрямком є **паралелізація обчислень.** Оскільки розрахунок локальних похибок MSE для кожного трикутника є незалежним, перенесення цього етапу на архітектуру багатопотокових обчислень на рівні графічного процесора (GPU) дозволить скоротити час обробки високороздільних зображень у десятки разів.

ПРИКІНЦЕВІ ПОЛОЖЕННЯ

Декларація про використання генеративного штучного інтелекту

У ході виконання кваліфікаційної роботи для окремих задач було використано системи генеративного штучного інтелекту (ГШІ). Використання даного інструментарію здійснювалося виключно в межах допустимих академічних стандартів та нормативних вимог до підготовки наукових робіт.

Генеративний штучний інтелект було застосовано для таких етапів підготовки матеріалів:

- **Стилізація та редагування тексту:** літературне редагування окремих фрагментів пояснювальної записки, виправлення стилістичних неточностей, а також автоматизована перевірка орфографічних, пунктуаційних та граматичних помилок в україномовному тексті.
- **Синтаксична оптимізація та розмітка:** автоматизація рутинних процесів верстки документа в системі комп'ютерного набору $\text{\LaTeX}$, зокрема оптимізація коду складних таблиць, налаштування параметрів пакетів відображення графічного матеріалу та форматування переліків.

Водночас декларується, що системи генеративного штучного інтелекту **не використовувалися** для написання основного наукового тексту роботи, формулювання теоретичних висновків, одержання чи моделювання результатів обчислювальних експериментів. Усі кількісні та якісні показники, наведені у практичній частині дослідження (зокрема, значення метрик $PSNR$, MSE , кількість обчислювальних точок, трикутників та часові характеристики), отримані виключно на основі роботи розробленого програмного комплексу на мові Python. Графічний матеріал, що ілюструє етапи адаптації обчислювальної сітки Делоне, побудований безпосередньо програмним кодом за реальними даними експериментів на вхідних тестових матрицях пікселів.

Вихідні коди

Усі вихідні коди розробленого програмного комплексу адаптивної тріангуляції Делоне завантажено в репозиторій Github: <https://github.com/kpm-lnu/2025-2026-RMP-42-bachelor-thesis>. Ключовим елементом структури є інтегрований файл документації `README.md`, який виступає головною інструкцією користувача. У цьому файлі детально зафіксовано архітектурні принципи функціонування адаптивного алгоритму, наведено вичерпний перелік технічних залежностей, а також подано покроковий посібник із налаштування, інсталяції необхідних пакетів та запуску обчислювального процесу в локальному середовищі.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Gonzalez R. C., Woods R. E. *Digital Image Processing*. 4th ed. New York: Pearson, 2018. 1192 p. ISBN 978-0-13-335672-4.
2. Sayood K. *Introduction to Data Compression*. 5th ed. Cambridge: Morgan Kaufmann, 2017. 790 p. DOI: <https://doi.org/10.1016/C2015-0-06248-7>.
3. Delaunay B. Sur la sphère vide // *Bulletin de l'Académie des Sciences de l'URSS*. 1934. Vol. 7. P. 793–800.
4. Shewchuk J. R. Triangle: Engineering a 2D Quality Mesh Generator and Delaunay Triangulator // *Applied Computational Geometry: Towards Geometric Engineering*. Lecture Notes in Computer Science. Berlin: Springer, 1996. Vol. 1148. P. 203–222. DOI: <https://doi.org/10.1007/BFb0014497>.
5. Ruppert J. A Delaunay Refinement Algorithm for Quality 2-Dimensional Mesh Generation // *Journal of Algorithms*. 1995. Vol. 18, № 3. P. 548–585. DOI: <https://doi.org/10.1006/jagm.1995.1021>.
6. Demaret L., Dyn N., Iske A. Image compression by linear splines over adaptive triangulations // *Signal Processing*. 2006. Vol. 86, № 7. P. 1604–1616. DOI: <https://doi.org/10.1016/j.sigpro.2005.09.003>.
7. Cohen-Steiner D., Alliez P., Desbrun M. Variational shape approximation // *ACM Transactions on Graphics*. 2004. Vol. 23, № 3. P. 905–914. DOI: <https://doi.org/10.1145/1015706.1015817>.
8. Wallace G. K. The JPEG still picture compression standard // *IEEE Transactions on Consumer Electronics*. 1992. Vol. 38, № 1. P. xviii–xxxiv. DOI: <https://doi.org/10.1109/30.125072>.
9. Harris C. R., Millman K. J., van der Walt S. J. et al. Array programming with NumPy // *Nature*. 2020. Vol. 585. P. 357–362. DOI: <https://doi.org/10.1038/s41586-020-2649-2>.
10. Van der Walt S., Schönberger J. L., Nunez-Iglesias J. et al. scikit-image: image processing in Python // *PeerJ*. 2014. Vol. 2. e453. DOI: <https://doi.org/10.7717/peerj.453>.
11. Bradski G. The OpenCV Library // *Dr. Dobb's Journal of Software Tools*. 2000. Vol. 25. P. 120–125.
12. Hunter J. D. Matplotlib: A 2D graphics environment // *Computing in Science & Engineering*. 2007. Vol. 9, № 3. P. 90–95. DOI: <https://doi.org/10.1109/MCSE.2007.55>.
13. Clark A. *Pillow (PIL Fork) Documentation* [Електронний ресурс]. Read the Docs, 2015. URL: <https://pillow.readthedocs.io> (дата звернення: 01.06.2026).
14. Shewchuk J. R. *Triangle: A Two-Dimensional Quality Mesh Generator and Delaunay Triangulator* [Електронний ресурс]. Carnegie Mellon University, 1996. URL: <https://www.cs.cmu.edu/~quake/triangle.html> (дата звернення: 01.06.2026).